Project Name: LoadBear

Team Members:

1. Achmad Jihad
2. Oktavian Alle Mahenswa Putra

Problem (The "Why")

On June 29, 2026, Mas Raka shared a story at a bootcamp sharing session. He said it took him two months to refactor a single method, not because the logic was complex, but because the person who originally built it had already resigned.

- **Current Issue:**

  Important work knowledge often lives in only one person's head. Right now there's no system to track who holds this kind of knowledge alone. The problem only shows up after that person resigns, gets sick, or switches teams, when something breaks and no one else knows how to fix it.

- **Impact:**

  This affects engineering managers, who lose time and client trust when expertise suddenly disappears. It affects HR, who can't plan training or succession ahead of time because they don't know where the risk is. And it affects the whole company through slower fixes, repeated onboarding costs, and client relationships that have to start over.

Proposed Solution

LoadBear helps teams see who's "load-bearing", the people holding up critical knowledge that no one else has.

- **Our Approach:**

  Teams log "knowledge items", a process, a client relationship, a script, a system. Each item has an owner, a note on whether someone else can also

do it, and two risk factors, how bad it would be to lose that knowledge, and how likely the owner is to leave soon. Then, LoadBear uses these to calculate a risk score and rank items, so managers can quickly see which few things actually need attention instead of chasing a huge documentation backlog.

- **Key Value:**

  This solution doesn't force people to write full documentation, which most people avoid doing anyway. Instead, it shows exactly where the risk is concentrated, so the fix can stay simple: pairing someone up, shadowing, or recording a short walkthrough instead of writing a manual.

Core Features

- **Feature 1:**

  Knowledge item registry. Add and edit items with owner, backup status, impact, and departure risk.

- **Feature 2:**

  Risk scoring. Auto calculates a score from impact, departure risk, and backup coverage, then ranks items high to low.

- **Feature 3:**

  Risk dashboard. Shows total items, high risk count, and coverage rate in one view.

High-Level Architecture & Quality Assurance

- **Architecture Pattern:**

  3-layer setup. Frontend (React), backend (.NET), database (MongoDB).

- **Design Patterns Used:**

  MVC for backend structure. Basic CRUD functions for managing items.

- **Code Quality Goal:**

Clean, readable code with clear naming. Unit tests on the risk scoring logic using Jest. Peer review before every merge.

Identified Challenges & Risks

- **Risk 1:**

  Getting honest input without it feeling like surveillance. Framing this as support, not blame, matters as much as the code.

- **Risk 2:**

  Keeping the risk formula simple while still being accurate. A "low impact" item could actually be tied to a major client.

- **Risk 3:**

  The risk formula is based on assumptions, not real data, so early scores may feel off until tuned.

- **Risk 4:**

  MongoDB's flexible schema could let incomplete data slip in without proper validation.